



武汉大学
WUHAN UNIVERSITY

公钥密码学实验

宁建廷

单位：武汉大学·国家网络安全学院

电子邮箱：jtning@whu.edu.cn

实验三：属性基加密算法实现与测试



01 实验目的及要求

02 Waters11算法的实现与测试

03 FABEO算法的实现与测试

目 录

CONTENTS



01

实验目的及要求





实验目的

- 熟悉经典属性基加密算法Waters11的核心底层逻辑和运算流程。
- 熟悉经典属性基加密算法FABEO的核心底层逻辑和运算流程。
- 提升密码算法工程实现能力。



实验要求

➤ 编程实现Waters11算法:

- ✓ 实现Waters11加解密算法涉及的辅助函数：密码杂凑函数和随机数发生器;
- ✓ 实现Waters11加解密算法：密钥生成算法Waters11_keygen、加密算法Waters11_Encrypt、解密算法Waters11_Decrypt;
- ✓ 检验Waters11加解密算法实现是否正确。

➤ 编程实现FABEO算法:

- ✓ 实现FABEO加解密算法涉及的辅助函数：密码杂凑函数和随机数发生器;
- ✓ 实现FABEO加解密算法：密钥生成算法FABEO_keygen、加密算法FABEO_Encrypt、解密算法FABEO_Decrypt;
- ✓ 检验FABEO加解密算法实现是否正确。



算法实现相关接口

本实验的算法实现依赖 **Charm-Crypto** 库，主要用到的核心模块与接口如下：

➤ 配对群生成接口：

- ✓ 调用路径：from charm.toolbox.pairinggroup import PairingGroup, ZR, G1, G2, GT, pair;
- ✓ 核心函数：groupObj = PairingGroup('SS512');
- ✓ 功能描述：调用此接口生成双线性配对群及相关群参数。通过调用核心函数可生成配对群，并使用群上操作。

➤ 随机数生成接口：

- ✓ 调用路径：from charm.toolbox.pairinggroup import PairingGroup, ZR, G1, G2, GT, pair;
- ✓ 核心函数：group.random()。

Charm-Crypto 库参考文档： <https://jhuisi.github.io/charm/>

02 | Waters11算法的实现



Waters11属性基加密算法——准备

➤ 系统参数:

- ✓ 设置属性基加密系统的最大属性数量 U ;
- ✓ 选择素数阶为 p 的群 $\mathbb{G}$, 以及群 $\mathbb{G}$ 的生成元 g ;
- ✓ 随机选择 U 个群元素 $h_1, \dots, h_U \in \mathbb{G}$;
- ✓ 随机选择 $\alpha, a \in \mathbb{Z}_p$;
- ✓ 设置公钥为 $PK = (g, e(g, g)^\alpha, g^a, h_1, \dots, h_U)$,
系统主私钥 $MSK = g^\alpha$;

➤ 辅助函数: Waters11属性基加密算法涉及一类辅助函数: 随机数发生器。



Waters11属性基加密算法——私钥生成

- 根据用户属性集 S ，为用户生成私钥 SK ，私钥生成过程如下：

Waters11.Keygen(PK, MSK, S):

输入：系统公钥 PK ，主私钥 MSK ，用户属性集 S

输出：用户私钥 SK

1: 随机选择 $t \in \mathbb{Z}_p$

2: 计算 $K = g^\alpha g^{at}$

3: 计算 $L = g^t$

4: $\forall x \in S$ ，计算 $K_x = h_x^t$

5: 用户私钥 $SK = (K, L, \{K_x\}_{x \in S})$



Waters11属性基加密算法——加密算法

➤ 设需要发送的明文为 M 。加密过程如下：

Waters11.Encrypt($PK, M, (M, \rho)$):

输入：公钥 PK ，明文 M ，访问结构 (M, ρ)

输出：密文 CT

- 1: 随机选择一个向量 $\mathbf{v} = (s, y_2, \dots, y_n) \in \mathbb{Z}_p^n$;
 - 2: 对于 $i \in \{1, l\}$, 计算 $\lambda_i = \mathbf{v} \cdot \mathbf{M}_i$, 其中 $\mathbf{M}_i$ 是对应于访问矩阵 $\mathbf{M}$ 第 i 行的向量;
 - 3: 随机选择 $r_1, \dots, r_l \in \mathbb{Z}_p$;
 - 4: 计算 $C = Me(g, g)^{\alpha s}$;
 - 5: 计算 $C' = g^s$;
 - 6: 计算 $(C_1 = g^{a\lambda_1} h_{\rho(1)}^{-r_1}, D_1 = g^{r_1}), \dots, (C_l = g^{a\lambda_l} h_{\rho(l)}^{-r_l}, D_l = g^{r_l})$
 - 7: 输出密文 $CT = (C, C', (C_1, D_1), \dots, (C_l, D_l))$
-



Waters11属性基加密算法——解密算法

➤ 接收者使用私钥 SK 对密文 CT 解密，解密过程如下：

Waters11.Decrypt(CT, SK):

输入：密文 CT ，私钥 SK

输出：明文 M

- 1: 假设属性集 S 满足密文中的访问结构(M, ρ)，令 $I \subset \{1, 2, \dots, l\}$ 定义为 $I = \{i: \rho(i) \in S\}$;
- 2: 设 $\{\omega_i \in \mathbb{Z}_p\}_{i \in I}$ 为一组常数，使得如果 λ_i 是根据 M 对任意秘密 s 的有效份额，则有 $\sum_{i \in I} \omega_i \lambda_i = s'$ （可能存在不同的方式来选择 ω_i 的值以满足此条件）；
- 3: 计算

$$\frac{e(C', K)}{\prod_{i \in I} \left(e(C_i, L) e(D_i, K_{\rho(i)}) \right)^{\omega_i}} = \frac{e(g, g)^{\alpha s} e(g, g)^{\alpha s t}}{\prod_{i \in I} e(g, g)^{t \alpha \lambda_i \omega_i}} = e(g, g)^{\alpha s}$$

- 4: 计算明文 $M = C / e(g, g)^{\alpha s}$;
 - 5: 输出明文 M ;
-



Waters11属性基加密算法伪代码

Waters11_Setup(U)

输入：系统中支持的属性总数 U

输出：公共参数 PK ，主密钥 MSK

- $(G, G_T, e, p) \leftarrow \text{GenerateBilinearGroup}()$
 $g \leftarrow \text{RandomGenerator}(G)$
- $\alpha, a \leftarrow_R \mathbb{Z}_p$
- for $i \leftarrow 1$ to U do
 $h_i \leftarrow_R G$
end for
- $PK \leftarrow (g, e(g, g)^\alpha, g^a, h_1, \dots, h_U), MSK \leftarrow g^\alpha$
- return (PK, MSK)



Waters11属性基加密算法伪代码

```
def setup(self):
    """
    Generates public key and master secret key.
    """

    if debug:
        print('Setup algorithm:\n')

    # pick a random element each from two source groups and pair them
    g1 = self.group.random(G1)
    g2 = self.group.random(G2)
    alpha = self.group.random(ZR)
    g1_alpha = g1 ** alpha
    e_gg_alpha = pair(g1_alpha, g2)

    a = self.group.random(ZR)
    g1_a = g1 ** a

    h = [0]
    for i in range(self.uni_size):
        h.append(self.group.random(G1))

    pk = {'g1': g1, 'g2': g2, 'g1_a': g1_a, 'h': h, 'e_gg_alpha': e_gg_alpha}
    msk = {'g1_alpha': g1_alpha}
    return pk, msk
```



Waters11属性基加密算法伪代码

Waters11_Keygen(PK, MSK, S)

输入：公钥PK，主私钥MSK，用户属性集 S

输出：用户私钥 SK

- $t \leftarrow_R \mathbb{Z}_p$, $K \leftarrow \text{MSK} * g^{(a * t)}$, $L \leftarrow g^t$
- 初始化数组 K_x
- for 每个元素 $x \in S$ do
 - $K_x[x] \leftarrow (h_x)^t$
- end for
- $\text{SK} \leftarrow (K, L, \{K_x[x]\}_{x \in S})$
- return SK



Waters11属性基加密算法伪代码

```
def keygen(self, pk, msk, attr_list):  
    """  
    Generate a key for a set of attributes.  
    """  
  
    if debug:  
        print('Key generation algorithm:\n')  
  
    t = self.group.random(ZR)  
    k0 = msk['g1_alpha'] * (pk['g1_a'] ** t)  
    L = pk['g2'] ** t  
  
    K = {}  
    for attr in attr_list:  
        K[attr] = pk['h'][int(attr)] ** t  
  
    return {'attr_list': attr_list, 'k0': k0, 'L': L, 'K': K}
```



Waters11属性基加密算法伪代码

Waters11_Encrypt(PK, ($\mathbf{M}, \rho$), M)

输入：公钥PK, 访问结构 ($\mathbf{M}, \rho$) , 明文消息 M

输出：密文 CT

- $s, y_2, \dots, y_n \leftarrow_R \mathbb{Z}_p, v \leftarrow (s, y_2, \dots, y_n)^T$
- $C \leftarrow M * e(g, g)^{(\alpha * s)} \quad C' \leftarrow g^s$
- for $i \leftarrow 1$ to l do
 - $\lambda_i \leftarrow M_i \cdot v$
 - $r_i \leftarrow_R \mathbb{Z}_p$
 - $C_i \leftarrow g^{(a * \lambda_i) * (h_{\rho(i)})^{(-r_i)}}$
 - $D_i \leftarrow g^{(r_i)}$
- end for
- $CT \leftarrow (C, C', \{(C_i, D_i)\}_{i=1 \text{ to } l}, M, \rho)$
- return CT



Waters11属性基加密算法伪代码

```
def encrypt(self, pk, msg, policy_str):
    """
    Encrypt a message M under a monotone span program.
    """

    if debug:
        print('Encryption algorithm:\n')

    policy = self.util.createPolicy(policy_str)
    mono_span_prog = self.util.convert_policy_to_msp(policy)
    num_cols = self.util.len_longest_row

    # pick randomness
    u = []
    for i in range(num_cols):
        rand = self.group.random(ZR)
        u.append(rand)
    s = u[0] # shared secret

    c0 = pk['g2'] ** s

    C = {}
    D = {}
    for attr, row in mono_span_prog.items():
        cols = len(row)
        sum = 0
        for i in range(cols):
            sum += row[i] * u[i]
        attr_stripped = self.util.strip_index(attr)
        r_attr = self.group.random(ZR)
        c_attr = (pk['g1_a'] ** sum) / (pk['h'][int(attr_stripped)] ** r_attr)
        d_attr = pk['g2'] ** r_attr
        C[attr] = c_attr
        D[attr] = d_attr

    c_m = (pk['e_gg_alpha'] ** s) * msg

    return {'policy': policy, 'c0': c0, 'C': C, 'D': D, 'c_m': c_m}
```



Waters11属性基加密算法伪代码

Waters11_Decrypt(PK, CT, SK)

输入：公钥PK, 密文 CT, 用户私钥 SK

输出：明文消息 M

- if S does not satisfy M then
 return $\perp$
 end if
- $I \leftarrow \{i \in \{1, \dots, l\} \mid \rho(i) \in S\}$
- Find constants $\{\omega_i \in \mathbb{Z}_p \mid i \in I\}$ such that:
 $\sum_{i \in I} (\omega_i * M_i) = (1, 0, \dots, 0)$
- Numerator $\leftarrow e(C', K)$, Denominator $\leftarrow 1$
- for each $i \in I$ do
 pairing_val $\leftarrow e(C_i, L) * e(D_i, K_{\rho(i)})$
 Denominator $\leftarrow$ Denominator * (pairing_val) $^{\omega_i}$
 end for
- PairingResult $\leftarrow$ Numerator / Denominator
- $M \leftarrow C /$ PairingResult
- return M



Waters11属性基加密算法伪代码

```
def decrypt(self, pk, ctxt, key):  
    """  
    Decrypt ciphertext ctxt with key key.  
    """  
  
    if debug:  
        print('Decryption algorithm:\n')  
  
    nodes = self.util.prune(ctxt['policy'], key['attr_list'])  
    if not nodes:  
        print ("Policy not satisfied.")  
        return None  
  
    prodG = 1  
    prodGT = 1  
  
    for node in nodes:  
        attr = node.getAttributeAndIndex()  
        attr_stripped = self.util.strip_index(attr)  
        prodG *= ctxt['C'][attr]  
        prodGT *= pair(key['K'][attr_stripped], ctxt['D'][attr])  
  
    return (ctxt['c_m'] * pair(prodG, key['L']) * prodGT) / (pair(key['k0'], ctxt['c0']))
```



03

FABEO算法的实现





FABEO属性基加密算法——准备

➤ 系统参数:

✓ 生成配对群参数 $\mathcal{G} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$;

✓ 选择随机数 $\alpha \in \mathbb{Z}_p$;

✓ 选择密码杂凑函数 $H: \mathcal{U} \rightarrow \mathbb{G}_1$;

✓ 主公钥 mpk 和主密钥 msk 如下:

$$mpk = (\mathcal{G}, H, e(g_1, g_2)^\alpha)$$

$$msk = \alpha$$

➤ 辅助函数: FABEO属性基加密算法涉及两类辅助函数: 密码杂凑函数和随机数发生器。



FABEO属性基加密算法——私钥生成

- 根据用户属性集 S ，为用户生成私钥 SK ，私钥生成过程如下：

FABEO.Keygen($mpk, msk, (M, \pi)$):

输入：系统公钥 mpk ，主私钥 msk ，访问结构 (M, π)

输出：用户私钥 sk

1: 随机选择 $r' \in \mathbb{Z}_p$

2: 随机选择 $v \in \mathbb{Z}_p^{n_2-1}$

2: 计算 $sk_1 = g_2^r$

3: 对于访问矩阵 M 的每一行 $i \in [n_1]$ ，计算 $sk_{2,i} = g^{M_i(a||v)^T} \cdot H(\pi(i))^r$

4: 用户私钥 $sk = (sk_1, \{sk_{2,i}\}_{i \in [n_1]})$



FABEO属性基加密算法——加密算法

➤ 设需要加密的明文为 M ，加密过程如下：

FABEO.Encrypt(mpk, M, S):

输入：主公钥 mpk ，明文 M ，明文属性 S

输出：密文 ct

1: 随机选取 $s \leftarrow \mathbb{Z}_p$

2: 对于集合 S 中的每一个属性 $u \in S$ ，计算 $ct_{1,u} = H(u)^s$

3: 计算 $ct_2 = g_2^s$

4: 计算 $ct_3 = M \cdot e(g_1, g_2)^{\alpha s}$

5: 输出密文 $ct = ((ct_{1,u})_{u \in S}, ct_2, ct_3)$



FABEO属性基加密算法——解密算法

➤ 接收者使用私钥 sk 对密文 ct 解密，解密过程如下：

FABEO.Decrypt(ct, sk):

输入：密文 ct , 私钥 sk

输出：明文 M

- 1: 假设属性集 S 满足密文中的访问策略 (M, ρ) , 令 $I \subset \{1, 2, \dots, l\}$ 定义为 $I = \{i: \rho(i) \in S\}$
- 2: 设 $\{\gamma_i \in \mathbb{Z}_p\}_{i \in I}$ 为一组常数, 使得如果 M_i 是根据 M 对任意秘密 s 的有效份额, 则有 $\sum_{i \in I} \gamma_i M_i = (1, 0, \dots, 0)$
- 3: 计算明文 M

$$M = \frac{e\left(\prod_{i \in I} (ct_{1, \pi(i)})^{\gamma_i}, sk_1\right) \cdot ct_3}{e\left(\prod_{i \in I} (sk_{2, i})^{\gamma_i}, ct_2\right)}$$

- 4: 输出明文 M
-



FABEO属性基加密算法伪代码

FABEO_Setup(1^λ)

输入：安全参数 1^λ

输出：公私钥对 (mpk, msk)

- // 1. 初始化双线性群和生成元
 $(G_1, G_2, G_T, e, p, g_1, g_2) \leftarrow \text{GroupGen}(1^\lambda)$
- // 2. 选取主密钥和哈希函数
 $\alpha \leftarrow_{\mathcal{R}} \mathbb{Z}_p$ $H \leftarrow \text{HashFunction}(U \rightarrow G_1)$ // U 为系统中所有可能属性的全集
- // 3. 构造主公钥和主密钥
 $mpk \leftarrow (G_1, G_2, G_T, e, p, g_1, g_2, H, e(g_1, g_2)^\alpha)$
 $msk \leftarrow \alpha$
- return (mpk, msk)



FABEO属性基加密算法伪代码

```
def setup(self):  
    """  
    Generates public key and master secret key.  
    """  
  
    if debug:  
        print('\nSetup algorithm:\n')  
  
    # pick a random element from the two source groups and pair them  
    g1 = self.group.random(G1)  
    g2 = self.group.random(G2)  
    e_g1g2 = pair(g1, g2)  
  
    alpha = self.group.random(ZR)  
  
    # now compute various parts of the public parameters  
    e_g1g2_alpha = e_g1g2 ** alpha  
  
    # the master secret and public key  
    msk = {'alpha': alpha}  
    pk = {'g1': g1, 'g2': g2, 'e_g1g2_alpha': e_g1g2_alpha}  
  
    return pk, msk
```



FABEO属性基加密算法伪代码

FABEO_Keygen(mpk, msk, (M, π))

输入：主公钥mpk，主私钥msk，访问策略

略(M, π)

输出：用户私钥 SK

- // 1. 选取随机向量
 $r' \leftarrow_R (Z_p)$
 $v \leftarrow_R (Z_p)^{(n_2-1)}$
- // 2. 计算第一部分私钥组件
 $sk_1 \leftarrow (g_2)^{r'}$
- // 3. 计算第二部分私钥组件 (与矩阵的行相关)
 $w \leftarrow (\alpha \parallel v)^T$

Initialize array sk_2

for $i \leftarrow 1$ to n_1 do

InnerProduct $\leftarrow M_i \cdot w$ //

M_i 是矩阵第 i 行

HashTerm $\leftarrow H(\pi(i))^{r'}$ // $\rho(i)$

为该属性的出现次序

$sk_{\{2,i\}} \leftarrow (g_1)^{\text{InnerProduct} *}$

HashTerm

end for

- $sk \leftarrow (\{sk_1, \{sk_{\{2,i\}}\}_{i=1 \text{ to } n_1}\})$
- return sk



FABEO属性基加密算法伪代码

```
def keygen(self, pk, msk, policy_str):
    """
    Generate a key for a list of attributes.
    """

    if debug:
        print('\nKey generation algorithm:\n')

    policy = self.util.createPolicy(policy_str)
    mono_span_prog = self.util.convert_policy_to_msp(policy)
    num_cols = self.util.len_longest_row

    # pick randomness
    r = self.group.random(ZR)

    g2_r = pk['g2'] ** r

    # pick random shares
    v = [msk['alpha']]
    for i in range(num_cols-1):
        rand = self.group.random(ZR)
        v.append(rand)

    sk = {}
    for attr, row in mono_span_prog.items():
        attr_stripped = self.util.strip_index(attr)
        attrHash = self.group.hash(attr_stripped, G1)
        len_row = len(row)
        Mivtop = sum(i[0] * i[1] for i in zip(row, v[:len_row]))
        sk[attr] = pk['g1'] ** Mivtop * attrHash ** r

    return {'policy': policy, 'g2_r': g2_r, 'sk': sk}
```



FABEO属性基加密算法伪代码

FABEO_Encrypt(mpk, S, M)

输入：主公钥mpk，密文属性 S ， 明文消息 M

输出：密文 ct

- // 1. 选取随机秘密值
 $s \leftarrow_{\mathcal{R}} \mathbb{Z}_p$
- // 2. 针对集合 S 中的每个属性生成密文组件
Initialize array ct_1
for each attribute $u \in S$ do
 $ct_{\{1,u\}} \leftarrow H(u)^s$
end for
- // 3. 生成共享的密文组件
 $ct_2 \leftarrow (g_2)^s$
- // 4. 加密明文
 $ct_3 \leftarrow M * (e(g_1, g_2)^{\alpha s})$
- $ct \leftarrow (\{ct_{\{1,u\}}\}_{u \in S}, ct_2, ct_3)$
- return ct



FABEO属性基加密算法伪代码

```
def encrypt(self, pk, msg, attr_list):
    """
    Encrypt a message msg under a policy string.
    """

    if debug:
        print('\nEncryption algorithm:\n')

    s = self.group.random(ZR)
    g2_s = pk['g2'] ** s

    ct = {}
    for attr in attr_list:
        attrHash = self.group.hash(attr, G1)
        ct[attr] = attrHash ** s

    # compute the e(g, h)^(As) * m term
    Cp = pk['e_g1g2_alpha'] ** s
    Cp = Cp * msg

    return {'attr_list': attr_list, 'g2_s': g2_s, 'ct': ct, 'Cp': Cp}
```



FABEO属性基加密算法伪代码

FABEO_Decrypt(mpk, ct, sk)

输入：主公钥mpk，密文 ct，私钥 sk

输出：明文消息 M

- // 1. 检查属性集合 S 是否满足访问矩阵 M if S does not satisfy (M, π) then
return $\perp$ end if
- // 2. 选取满足条件的行索引集合 I，并计算线性重构系数
 $I \leftarrow \{i \in \{1, \dots, n-1\} \mid \pi(i) \in S\}$
Find constants $\{\gamma_i \in \mathbb{Z}_p\}_{i \in I}$ such that:
 $\sum_{i \in I} (\gamma_i * M_i) = (1, 0, \dots, 0)$
- // 3. 计算配对公式的分子
Numerator_Base $\leftarrow 1$
for each $i \in I$ do

Numerator_Base $\leftarrow$ Numerator_Base *

$(sk_{\{2,i\}})^{(\gamma_i)}$ end for

Numerator $\leftarrow e(\text{Numerator_Base}, ct_2)$

- // 4. 计算配对公式的分母部分

Denominator $\leftarrow 1$

for $j \leftarrow 1$ to τ do

$I_j \leftarrow \{i \in I \mid \rho(i) = j\}$

Term_j_Base $\leftarrow 1$

for each $i \in I_j$ do

Term_j_Base $\leftarrow$ Term_j_Base *

$(ct_{\{1,\pi(i)\}})^{(\gamma_i)}$ end for

Term_j $\leftarrow e(\text{Term_j_Base}, sk_{\{1,j\}})$

Denominator $\leftarrow$ Denominator * Term_j end for

- // 5. 恢复封装的密钥

$d \leftarrow \text{Numerator} / \text{Denominator}$

- return d



FABEO属性基加密算法伪代码

```
def decrypt(self, pk, ctxt, key):  
    """  
    Decrypt ciphertext ctxt with key key.  
    """  
  
    if debug:  
        print('\nDecryption algorithm:\n')  
  
    nodes = self.util.prune(key['policy'], ctxt['attr_list'])  
  
    if not nodes:  
        print ("Policy not satisfied.")  
        return None  
  
    prod_sk = 1  
    prod_ct = 1  
    for node in nodes:  
        attr = node.getAttributeAndIndex()  
        attr_stripped = self.util.strip_index(attr) # no need, re-use not allowed  
  
        prod_sk *= key['sk'][attr]  
        prod_ct *= ctxt['ct'][attr_stripped]  
  
    e1 = pair(prod_sk, ctxt['g2_s'])  
    e2 = pair(prod_ct, key['g2_r'])  
  
    kem = e1/e2  
  
    return ctxt['Cp'] / kem
```


谢谢！